

PEDOMAN PENGGUNAAN AGENT

UNTUK PENGEMBANGAN AWCMS, AWCMS-MINI, AWCMS-MICRO, DAN SOFTWARE TURUNANNYA

*Standar orkestrasi AI engineering yang aman, atomic, teruji,
dan selaras dengan arsitektur keluarga AWCMS*

AWCMS

ERP, multi-bisnis, proses lintas unit, integrasi enterprise

AWCMS-Mini

Modular monolith berukuran lebih kecil, fondasi aplikasi terarah

AWCMS-Micro

Full-online website dan digitalisasi bisnis skala lebih kecil

Versi 1.0 | 15 Juli 2026

PT Ahli Web Internasional - AhliWeb Ecosystem

Kendali Dokumen

Atribut	Keterangan
Judul	Pedoman Penggunaan Agent untuk Pengembangan AWCMS, AWCMS-Mini, AWCMS-Micro, dan Software Turunannya
Versi	1.0
Status	Pedoman operasional dan acuan implementasi
Pemilik	PT Ahli Web Internasional / AhliWeb Ecosystem
Ruang lingkup	Perencanaan, coding, review, security audit, testing, release, deployment, dan pemeliharaan berbantuan agent AI
Basis teknis	Keluarga AWCMS mengadopsi fondasi yang sama dengan AWCMS-Mini; perbedaan utama berada pada skala, orientasi produk, dan kedalaman domain
Sumber utama	AWCMS-Mini README, AGENTS.md, CONTRIBUTING.md, derived-application-guide, project skills, dan subagent definitions

Catatan interpretasi

Pedoman ini menetapkan satu baseline engineering untuk keluarga AWCMS. Implementasi aktual AWCMS dan AWCMS-Micro dapat memiliki repositori, modul, atau deployment profile yang berbeda. Ketika terdapat perbedaan, AGENTS.md, ADR, dan kontrak di masing-masing repository menjadi sumber kebenaran paling spesifik.

Daftar Isi

1. Ringkasan Eksekutif
2. Tujuan dan Ruang Lingkup
3. Posisi AWCMS, AWCMS-Mini, dan AWCMS-Micro
4. Prinsip Dasar Penggunaan Agent
5. Klasifikasi Pekerjaan Sebelum Coding
6. Peran Agent dan Tata Kelola
7. Alur Kerja End-to-End
8. Strategi Branch, Worktree, dan Paralelisme
9. Pedoman Implementasi per Platform
10. Testing, Review, dan Security Gate
11. Release, Deployment, dan Operasional
12. Larangan dan Stop Conditions
13. Template Prompt Agent
14. Checklist Operasional
15. Contoh Penerapan
16. Pelajaran dari Pengembangan AWCMS-Mini
17. Standar dan Kepatuhan
18. Penutup
- Lampiran A. Format Laporan Agent
- Lampiran B. Matriks Pemilihan Skill
- Referensi

1. Ringkasan Eksekutif

Pedoman ini menetapkan cara menggunakan agent AI sebagai bagian dari proses rekayasa perangkat lunak keluarga AWCMS. Agent diposisikan sebagai tenaga engineering yang bekerja di bawah kontrak repository, issue, acceptance criteria, quality gate, dan persetujuan maintainer - bukan sebagai pengganti tata kelola pengembangan.

Pola inti

Satu kebutuhan -> satu issue atomic -> satu branch/worktree -> satu coder utama -> reviewer independen -> security auditor sesuai risiko -> validasi penuh -> satu pull request -> persetujuan maintainer -> release dan deployment terkontrol.

AWCMS-Mini telah membuktikan pola ini melalui kontrak AGENTS.md, skill tingkat proyek, subagent coder/reviewer/security auditor, migration berurutan, kontrak OpenAPI/AsyncAPI, test berlapis, generated inventory, compatibility manifest, CI, production preflight, performance suite, dan disaster-recovery drill. Pedoman ini memperluas pola tersebut agar konsisten digunakan pada AWCMS, AWCMS-Mini, AWCMS-Micro, dan seluruh software turunannya.

- **AWCMS:** orientasi ERP dan multi-bisnis, proses lintas legal entity/unit, integrasi enterprise, governance lebih ketat.
- **AWCMS-Mini:** scope lebih kecil daripada AWCMS, tetapi tetap modular, aman, tenant-aware, dan siap menjadi fondasi aplikasi turunan.
- **AWCMS-Micro:** orientasi full-online website, portal, layanan digital, dan bisnis berskala lebih kecil dengan implementasi lebih ringkas.
- **Software turunan:** menambahkan domain aplikasi tanpa membangun ulang fondasi identity, access control, RLS, audit, logging, idempotency, i18n, atau deployment tooling.

2. Tujuan dan Ruang Lingkup

2.1 Tujuan

1. Menyeragamkan penggunaan agent pada seluruh keluarga AWCMS.
2. Mencegah agent membangun ulang fitur dasar yang sudah tersedia.
3. Menjaga setiap perubahan tetap atomic, dapat direview, diuji, dan dipulihkan.
4. Memisahkan tanggung jawab coder, reviewer, security auditor, dan maintainer.
5. Menjamin dokumentasi, kontrak, migration, test, dan implementasi selalu sinkron.
6. Menerjemahkan prinsip keamanan dan kepatuhan menjadi langkah teknis nyata.

2.2 Ruang lingkup

- Analisis kebutuhan dan klasifikasi lapisan produk.
- Penyusunan epic, wave, dan issue atomic.
- Pengelolaan branch, worktree, dan multi-agent.
- Implementasi database, domain logic, API, event, UI, integrasi, dan dokumentasi.
- Unit, integration, contract, security, browser E2E, performance, resilience, dan deployment testing.
- Pull request, review, security audit, release, deployment, monitoring, dan laporan hasil.

2.3 Di luar ruang lingkup

- Mengizinkan agent mengubah production tanpa approval.
- Mengizinkan beberapa agent menulis file yang sama tanpa koordinasi.

- Menggunakan agent untuk memproses secret atau data pelanggan asli secara sembarangan.
- Menganggap output agent benar tanpa test dan review independen.

3. Posisi AWCMS, AWCMS-Mini, dan AWCMS-Micro

Ketiga platform menggunakan prinsip fondasi yang sama: arsitektur modular, kontrak modul, keamanan default-deny, PostgreSQL dengan RLS untuk data tenant-scoped, API/event contract, auditability, testability, dokumentasi sinkron, dan deployment yang dapat diverifikasi. Perbedaannya terutama pada orientasi produk, skala organisasi, kedalaman proses, serta profil deployment.

Dimensi	AWCMS	AWCMS-Mini	AWCMS-Micro
Orientasi utama	ERP, multi-bisnis, proses enterprise	Aplikasi modular dengan scope menengah/kecil	Website, portal, layanan full-online dan bisnis lebih kecil
Kompleksitas domain	Tinggi: finance, inventory, sales, procurement, HR, tax, workflow lintas unit	Sedang: modul terarah sesuai kebutuhan aplikasi	Rendah-sedang: content, commerce ringan, form, portal, CRM ringan
Skala organisasi	Multi legal entity, multi unit, multi tenant, multi lokasi	Single-tenant default dengan tenant-aware capability	Single site/brand/tenant dominan, dapat berkembang menjadi multi-site
Integrasi	ERP, payment, government, data warehouse, enterprise SSO	Provider dan sistem domain melalui ports/outbox	Cloudflare, CMS, forms, CRM, payment ringan, social/email
Mode operasi	Online, hybrid, LAN, atau multi-site sesuai proses bisnis	Offline/LAN-first dengan optional online sync	Full-online sebagai orientasi utama
Governance agent	Paling ketat; ADR, SoD, accounting/period controls, performance dan DR	Ketat dan reusable; mengikuti AGENTS.md dan skill AWCMS-Mini	Lebih ringkas tetapi tidak mengurangi security, test, atau review

3.1 Baseline keluarga AWCMS

- Bun runtime dan tooling Bun-only, kecuali pengecualian terdokumentasi.
- Astro sebagai web framework dan server-rendered application foundation.
- PostgreSQL dengan migration berurutan, least-privilege role, dan RLS.
- Modular monolith dengan capability contract dan batas modul yang eksplisit.
- RBAC + ABAC default-deny, audit log, correlation ID, dan redaction.
- OpenAPI untuk REST dan AsyncAPI untuk domain event.
- Idempotency untuk mutation berisiko tinggi.
- Soft delete untuk master/config/draft dan immutability untuk posted data.
- Provider eksternal di luar transaksi kritikal melalui outbox/queue.
- CI, generated inventory, test berlapis, preflight, backup, dan rollback.

4. Prinsip Dasar Penggunaan Agent

Prinsip	Ketentuan operasional
Issue sebagai unit kerja	Agent hanya mengerjakan kebutuhan yang telah memiliki issue, scope, acceptance criteria, dan dependensi yang jelas.
Satu writer utama	Satu branch/worktree memiliki satu coder utama. Reviewer dan auditor bersifat read-only.

Prinsip	Ketentuan operasional
Inspect before edit	Agent membaca AGENTS.md, README, docs, migration, module, route, OpenAPI, AsyncAPI, test, dan histori PR sebelum mengubah kode.
Reuse before build	Agent wajib mencari capability base yang tersedia sebelum membuat implementasi baru.
Atomic delivery	Satu PR menyelesaikan satu issue dan tidak memuat unrelated change.
Security by default	Auth, tenant context, ABAC, RLS, validation, idempotency, audit, masking, dan error safety diterapkan pada jalur request.
Test as evidence	Status selesai harus dibuktikan dengan command dan test result, bukan narasi agent.
Docs as contract	Dokumentasi, migration, API/event contract, UI strings, generated inventory, dan changeset mengikuti implementasi nyata.
Maintainer gate	Agent tidak melakukan merge atau deployment sensitif tanpa persetujuan manusia yang berwenang.
Truthful reporting	Agent harus menyatakan command yang gagal, test yang tidak dijalankan, residual risk, dan keterbatasan.

5. Klasifikasi Pekerjaan Sebelum Coding

Klasifikasi menentukan repository, kontrak, agent, dan quality gate yang tepat. Coding tidak dimulai sebelum klasifikasi disepakati.

Kelas pekerjaan	Ditempatkan di	Kriteria utama	Contoh
Core maintenance	Repository platform	Perbaikan bug, compatibility, reliability, security, atau tooling generik	Fix RLS helper, idempotency utility, migration runner
Core hardening	Repository platform	Audit dan peningkatan kontrol existing	Redaction, query plan, DR, CodeQL
Generic optional module	Platform bila admission disetujui	Provider-neutral, reusable lintas aplikasi, dapat dinonaktifkan	Reference data, document infrastructure, integration hub
Derived application	Repository software turunan	Proses bisnis khusus aplikasi/industri	Kasir, sekolah, kesehatan, surat tugas, marketplace
SaaS control plane	Repository terpisah	Provisioning, subscription, billing, quota, reseller, lifecycle tenant	Tenant billing dan domain provisioning
ERP extension	Repository AWCMS/ERP extension	Ledger, posting, period lock, valuation, AR/AP, payroll, tax	Accounting engine dan inventory valuation
Documentation/tooling	Repository yang memiliki kontrak	Tidak menambah domain behavior	Update skill, generated inventory, CI check

5.1 Pertanyaan keputusan

- [] Apakah capability dibutuhkan lebih dari satu aplikasi?
- [] Apakah capability provider-neutral dan tidak membawa aturan bisnis satu industri?
- [] Apakah base telah menyediakan capability serupa?
- [] Apakah perubahan memerlukan ADR atau module admission?
- [] Apakah kebutuhan termasuk ERP, SaaS control plane, atau derived application?
- [] Apakah issue dapat selesai dalam satu PR atomic?
- [] Apakah dependensi issue sudah tersedia?

6. Peran Agent dan Tata Kelola

Peran	Hak akses	Tanggung jawab	Output utama
Orchestrator/Maintainer	Read + keputusan write/merge	Klasifikasi, issue readiness, branch/worktree, dependency, approval	Scope, assignment, merge/deploy decision
Planner/Architect	Read-only	Membaca repo, membuat impact map, architecture/security plan	Implementation plan dan risk map
Coder	Writer pada satu branch	Implementasi end-to-end, test, docs, changeset, laporan	Code dan evidence
Reviewer	Read-only	Review diff terhadap issue dan Definition of Done	Approve/Request changes/Comment
Security Auditor	Read-only	Threat surface, adversarial review, go-live verdict	PASS/BLOCKED + severity findings
Test/UX/Performance Analyst	Read-only atau worktree terpisah	Scenario, accessibility, performance, test gap	Rekomendasi dan evidence tambahan

6.1 Model tiga agent inti

```

GitHub Issue
-> awcms-* -coder (writer)
    -> awcms-* -reviewer (read-only)
        -> awcms-* -security-auditor (read-only, bila sensitif)
            -> maintainer approval
                -> merge / release / deployment
  
```

Nama agent dapat disesuaikan dengan repository, misalnya awcms-coder, awcms-mini-coder, atau awcms-micro-coder. Kontrak peran harus tetap sama: coder menulis, reviewer dan auditor menilai secara independen.

7. Alur Kerja End-to-End

Fase 1 - Intake dan perencanaan

- 1. Terima kebutuhan.** Catat masalah, tujuan bisnis, pengguna, data, integrasi, target deployment, risiko, dan indikator keberhasilan.
- 2. Klasifikasikan lapisan.** Tentukan core, optional module, derived app, SaaS control plane, ERP extension, atau tooling.
- 3. Analisis repository.** Baca kontrak dan identifikasi implementasi existing yang dapat digunakan ulang.

4. **Susun dokumen proporsional.** Produk/epic besar memakai PRD/SRS/ERD/threat model; bug kecil cukup issue, reproduksi, regression test, dan docs terkait.
5. **Pecah menjadi epic-wave-issue.** Setiap issue memiliki acceptance criteria, dependency, test plan, security notes, dan rollback impact.

Fase 2 - Baseline dan workspace

```
git checkout main
git pull --ff-only
bun install --frozen-lockfile
bun run check
```

Kegagalan baseline dicatat sebagai pre-existing failure. Agent tidak boleh mengklaim failure tersebut disebabkan atau diperbaiki oleh issue tanpa bukti.

```
git checkout -b feature/<issue>-<short-name>
# atau
git worktree add ../worktree-<issue> -b feature/<issue>-<short-name> main
```

Fase 3 - Implementation plan

Sebelum edit, coder menyerahkan implementation plan yang berisi:

- Pemahaman masalah dan asumsi.
- Existing capability yang digunakan ulang.
- Scope dan out of scope.
- File yang diperkirakan berubah.
- Dampak schema, RLS, API, event, UI, i18n, config, dan deployment.
- Test scenarios termasuk adversarial test.
- Risiko, rollback, dan urutan eksekusi.

Fase 4 - Implementasi vertical slice

1. Domain rule dan pure function.
2. Validation dan types.
3. Migration PostgreSQL, constraints, index, dan RLS.
4. Infrastructure adapter/repository.
5. Application service dan transaction boundary.
6. Permission domain, ABAC, dan audit decision.
7. Idempotency untuk high-risk mutation.
8. REST route dan OpenAPI.
9. Domain event dan AsyncAPI.
10. UI, accessibility, dan i18n.
11. Unit, integration, contract, security, dan E2E test.
12. Documentation, generated inventory, dan changeset.

Fase 5 - Fast feedback

```
bun run typecheck
bun test tests/unit/<target>.test.ts
bun test tests/integration/<target>.integration.test.ts

# sesuai dampak
bun run db:migrate
bun run openapi:bundle
bun run api:docs:generate
bun run api:spec:check
bun run i18n:extract
bun run modules:compose:check
bun run extension:check
```

Targeted test dijalankan selama coding. Full gate dijalankan pada checkpoint sebelum PR dan sesudah perbaikan review yang material.

Fase 6 - Self-review dan laporan coder

```
git diff --check
git status
git diff main...HEAD
bun run db:migrate
bun run check
```

Coder melaporkan file, command, test result, security notes, docs updates, limitation, dan next step. Tidak boleh ada klaim sukses tanpa command evidence.

Fase 7 - Review independen

- Reviewer memeriksa scope, correctness, migration, contract, test, docs, dan changeset.
- Security auditor memeriksa threat surface, tenant isolation, idempotency, data exposure, provider boundary, race condition, dan abuse case.
- Reviewer dan auditor tidak mengedit kode; temuan kembali kepada coder.
- Critical dan High wajib selesai sebelum merge. Medium harus diperbaiki atau diterima sebagai residual risk yang terdokumentasi.

Fase 8 - PR, CI, dan merge

1. Tambahkan changeset bila behavior berubah.
2. Buka PR yang menautkan Closes #NNN.
3. Tunggu CI, reviewer approval, security PASS, dan semua thread selesai.
4. Pastikan branch terbaru terhadap main.
5. Maintainer melakukan merge; branch dihapus setelah merge aman.

Fase 9 - Release dan deployment

1. Verifikasi merge commit dan artifact/image immutable.
2. Deploy ke staging atau environment uji yang setara.
3. Jalankan migration, smoke test, critical journey, log/audit verification, dan rollback verification.
4. Jalankan production preflight, backup verification, dan approval.
5. Deploy production secara terkontrol.
6. Lakukan post-deploy health check, monitoring, dan laporan.

Fase 10 - Retrospective

- Catat bagian paling lama dan penyebabnya.
- Ubah recurring finding menjadi linter, skill, checklist, atau repo-wide issue.
- Pisahkan temuan unrelated menjadi issue baru.
- Perbarui AGENTS.md, ADR, docs, dan skill bila standar berubah.

8. Strategi Branch, Worktree, dan Paralelisme

Aturan utama

Satu branch/worktree hanya memiliki satu writer utama. Paralelisme dilakukan pada issue atau worktree yang berbeda, atau melalui agent read-only.

Pola	Aman bila	Tidak aman bila
Satu coder + reviewer	Issue atomic dan file ownership jelas	Reviewer mengedit langsung tanpa koordinasi
Coder + test analyst	Test analyst read-only atau branch terpisah	Keduanya mengubah fixture/file yang sama
Beberapa worktree	Issue tidak saling bergantung dan migration namespace dikoordinasikan	Dua worktree mengambil nomor migration yang sama
Epic multi-wave	Setiap issue memiliki PR sendiri dan dependency graph	Semua wave digabung dalam satu giant PR
Generated artifacts	Hanya satu integrator menjalankan generator final	Banyak agent mengubah bundle/inventory secara paralel

8.1 Maksimal lima agent

Agent	Peran	Mode
1	Planner/architect	Read-only
2	Coder utama	Writer branch issue
3	Test/UX/performance analyst	Read-only atau worktree terpisah
4	PR reviewer	Read-only
5	Security auditor	Read-only

9. Pedoman Implementasi per Platform

9.1 AWCMS - ERP dan multi-bisnis

- Wajib membedakan tenant, legal entity, organization unit, office, warehouse, dan business scope.
- Posting transaksi bersifat immutable; koreksi melalui reversal/adjustment.
- Gunakan period lock, segregation of duties, approval matrix, dan audit evidence.
- Integrasi finance, inventory, sales, purchase, payroll, tax, dan payment melalui kontrak dan domain events.
- Quality gate tambahan: ledger reconciliation, valuation consistency, concurrency/locking, financial report projection, DR full drill, performance soak, dan approval bisnis.
- Agent tidak boleh mengubah accounting rule tanpa traceability ke requirement dan acceptance example.

9.2 AWCMS-Mini - modular application foundation

- Pertahankan base generik; pekerjaan baru berupa maintenance/hardening atau aplikasi turunan.
- Gunakan skill proyek sebagai pola implementasi, bukan membuat struktur sendiri.
- Data tenant-scoped memakai tenant_id, ABAC, ENABLE + FORCE RLS, dan index tenant-first.
- Gunakan application registry dan extension manifest pada repository turunan.
- Gunakan offline/LAN-first bila proses operasional memerlukan continuity tanpa internet.
- Jalankan bun run check, extension:check, security:readiness, dan production:preflight sesuai risiko.

9.3 AWCMS-Micro - full-online website dan bisnis lebih kecil

- Prioritaskan kesederhanaan, waktu implementasi, SEO, accessibility, responsive UI, dan operasi full-online.
- Tetap gunakan auth, access control, audit, masking, rate limiting, Turnstile, secure upload, dan provider isolation sesuai kebutuhan.
- Pilih modul minimum; hindari membawa kompleksitas ERP atau offline sync bila tidak dibutuhkan.
- Cloudflare DNS/CDN/WAF/Turnstile/R2 dapat menjadi profil umum, namun provider tetap berada di belakang adapter dan konfigurasi environment.
- E2E browser, security headers, public-route testing, Core Web Vitals/performance, SEO metadata, backup, dan deployment rollback menjadi gate utama.
- Agent harus menilai apakah kebutuhan dapat diselesaikan sebagai konfigurasi/content/template sebelum menambah modul baru.

9.4 Software turunan

- Gunakan registry aplikasi sendiri; jangan mengedit registry base.
- Gunakan migration namespace aplikasi dan extension.manifest.json.
- Tambahkan permission, entity, endpoint, event, UI, dan policy domain tanpa menduplikasi core.
- Interaksi lintas repository menggunakan contract, capability port, API, event, atau read model; jangan shared-table write.
- Mulai modul sebagai experimental dan naikan menjadi active setelah dipakai, diuji, dan melalui security review.

10. Testing, Review, dan Security Gate

10.1 Piramida testing

Lapisan	Tujuan	Contoh
Unit	Membuktikan domain rule dan pure function	Validation, policy, calculation, state transition
Integration	Membuktikan service/route terhadap PostgreSQL nyata	Migration, RLS, transaction, repository
Contract	Menjaga route, OpenAPI, event, AsyncAPI, error schema	Spec check dan route parity
Security/adversarial	Membuktikan abuse case ditolak	Cross-tenant access, reused idempotency key, IDOR, replay
Browser E2E	Membuktikan critical user journey pada UI nyata	Login, form, approval, reporting, public page
Performance	Mencegah regresi query, load, soak, saturation	Query plan budget, throughput, noisy neighbor
Resilience/DR	Membuktikan backup, restore, failure recovery	Restore drill, provider outage, worker retry

10.2 Matriks gate berdasarkan perubahan

Jenis perubahan	Gate minimum	Gate tambahan
Docs-only	Format, link, docs check	A11y/navigation bila dokumen publik
UI/public website	Typecheck, unit, build	Browser E2E, a11y, SEO, responsive, security headers
Schema + API	Migration, integration, OpenAPI, check	RLS isolation, idempotency, rollback
Auth/access/PII	Integration + security tests	Independent audit, security readiness, adversarial tests
ERP posting	Transaction, locking, immutability	Reconciliation, period lock, SoD, performance
Provider/integration	Outbox/retry, config validation	Failure simulation, signature/replay, staging
Deployment	Build, config, health	Preflight, backup/restore, smoke, rollback

10.3 Severity dan keputusan

Severity	Keputusan
Critical	Wajib diperbaiki; memblokir merge dan go-live.
High	Wajib diperbaiki sebelum merge.
Medium	Perbaiki atau dokumentasikan residual risk dengan owner dan follow-up issue.
Low	Dapat menjadi follow-up bila tidak mempengaruhi acceptance criteria.
Informational	Catatan peningkatan atau dokumentasi.

11. Release, Deployment, dan Operasional

11.1 Pre-merge

- ☐ Scope issue terpenuhi tanpa unrelated change.
- ☐ Migration, OpenAPI, AsyncAPI, i18n, config, inventory, dan docs sinkron.
- ☐ Test relevan dan full quality gate lulus.
- ☐ Reviewer Approve dan security auditor PASS bila diperlukan.
- ☐ Changeset tersedia untuk perubahan behavior.
- ☐ Tidak ada secret, dump, backup, atau data pelanggan dalam diff.

11.2 Pre-production

```
bun run config:validate
bun run security:readiness
bun run production:preflight
```

- ☐ Artifact/image dibangun dari merge commit.
- ☐ Database runtime memakai least-privilege role.
- ☐ Backup telah dibuat dan restore pernah diverifikasi.

- [] Migration plan dan rollback strategy disetujui.
- [] Environment secret berasal dari secrets manager atau environment injection.
- [] Health check, logging, monitoring, alerting, dan worker schedule tersedia.
- [] Staging smoke test dan critical journey lulus.

11.3 Post-deployment

- Health endpoint dan database connectivity.
- Login, authorization, dan tenant isolation.
- Critical user journey.
- Audit event, structured log, dan redaction.
- Outbox/worker/provider queue.
- Performance dan error rate.
- Backup status dan rollback readiness.

12. Larangan dan Stop Conditions

12.1 Larangan

1. Mengimplementasikan issue yang dependensinya belum selesai.
2. Menjalankan beberapa coder pada branch yang sama.
3. Mengubah migration lama yang sudah dirilis.
4. Mengedit generated OpenAPI bundle atau inventory tanpa generator.
5. Menambah framework/runtime tanpa kebutuhan tervalidasi dan approval.
6. Membangun ulang auth, audit, RLS, idempotency, logging, atau sync yang sudah ada.
7. Menggunakan database superuser untuk runtime.
8. Memanggil provider eksternal di dalam database transaction.
9. Commit secret, .env, data pelanggan, dump, atau backup.
10. Menghapus posted/append-only entity.
11. Mengklaim command/test berhasil tanpa menjalankannya.
12. Merge atau deploy perubahan sensitif tanpa maintainer gate.

12.2 Stop conditions

Agent harus berhenti dan melapor bila:

Scope tidak jelas; issue tidak atomic; ditemukan secret; migration number konflik; dependency belum tersedia; baseline gagal; perubahan memerlukan keputusan ADR; test produksi berisiko merusak data; akses atau environment tidak cukup; atau hasil security audit berstatus BLOCKED.

13. Template Prompt Agent

13.1 Prompt orchestrator/planner

Analisis kebutuhan berikut untuk keluarga AWCMS.

1. Baca AGENTS.md, README, ADR, docs, module, migration, API/event contract, test, issue, dan PR terkait.
 2. Klasifikasikan: core, optional generic module, derived application, SaaS control plane, ERP extension, atau tooling.
 3. Identifikasi capability existing yang harus digunakan ulang.
 4. Susun architecture/security impact, dependency graph, wave, dan issue atomic.
 5. Untuk setiap issue tulis scope, out of scope, acceptance criteria, test plan, docs checklist, risiko, dan rollback impact.
- Jangan mengedit kode.

13.2 Prompt coder

Gunakan agent coder repository ini dan kerjakan Issue #NNN secara atomic. Sebelum mengedit, baca AGENTS.md, issue, docs/ADR, module, SQL, OpenAPI, AsyncAPI, test, dan PR terkait. Berikan implementation plan dahulu.

Ketentuan:

- satu branch/worktree, satu writer utama, tanpa unrelated change;
- gunakan capability base, jangan membangun ulang;
- schema berubah -> migration baru + RLS/index;
- API/event berubah -> OpenAPI/AsyncAPI;
- high-risk mutation -> idempotency terikat resource/action/body;
- high-risk action -> audit + redaction;
- provider eksternal di luar transaction melalui outbox/queue;
- unit, integration, contract, security/adversarial, dan E2E sesuai risiko;
- update docs, generated artifacts, i18n, config, dan changeset.

Jalankan command relevan. Akhiri dengan laporan evidence, limitation, dan next recommended step. Jangan klaim sukses bila command gagal.

13.3 Prompt reviewer

Review Issue #NNN dan PR #NNN secara read-only.

Periksa scope, unrelated change, migration, RLS, OpenAPI/AsyncAPI, auth, tenant context, ABAC, idempotency, audit, masking, soft delete, immutability, validation, error safety, tests, docs, generated artifacts, dan changeset.

Output: Verdict; Critical; Security; Functional; Data/Migration; API/Event Contract; Testing Gaps; Documentation Gaps; Suggested Patch. Jangan mengedit file.

13.4 Prompt security auditor

Audit PR/modul secara read-only. Petakan threat surface dan uji secara adversarial: auth bypass, cross-tenant access, ABAC/RLS, IDOR/BOLA, mass assignment, injection, SSRF, secret leakage, replay, idempotency, race condition, audit/PII leakage, upload, provider boundary, retention, backup, dan rollback.

Setiap temuan: severity, file:baris, skenario eksploitasi, dampak, rekomendasi, dan regression test. Verdict: PASS atau BLOCKED.

14. Checklist Operasional

14.1 Sebelum coding

- ☐ Issue tersedia, atomic, dan acceptance criteria terukur.
- ☐ Klasifikasi platform/lapisan telah disetujui.
- ☐ Dependency selesai.
- ☐ Main terbaru dan baseline gate diketahui.
- ☐ Branch/worktree dibuat.
- ☐ Satu coder utama ditetapkan.

- [] AGENTS.md, docs, code, migration, contract, dan test dibaca.
- [] Implementation plan tersedia.

14.2 Sebelum PR

- [] Implementation sesuai scope.
- [] Migration, RLS, constraints, dan index tersedia.
- [] OpenAPI/AsyncAPI dan generated docs sinkron.
- [] ABAC, idempotency, audit, masking, dan error handling diterapkan.
- [] Unit, integration, adversarial, dan E2E test lulus sesuai risiko.
- [] Generated inventory, i18n, config docs, dan changeset diperbarui.
- [] bun run check lulus atau failure dijelaskan dengan jujur.

14.3 Sebelum merge

- [] Reviewer Approve.
- [] Security auditor PASS bila diwajibkan.
- [] Critical/High nol.
- [] CI hijau dan review thread selesai.
- [] Branch sinkron dengan main.
- [] Rollback dan deployment notes tersedia.
- [] Maintainer menyetujui merge.

14.4 Sebelum production

- [] Artifact berasal dari merge commit.
- [] Config, security readiness, dan preflight lulus.
- [] Backup dan restore verification tersedia.
- [] Least-privilege runtime role dan secret injection benar.
- [] Staging critical journey lulus.
- [] Monitoring, alert, health check, worker, dan rollback siap.

15. Contoh Penerapan

Contoh 1 - AWCMS ERP: modul purchase-to-pay

Planner memecah vendor master, purchase request, approval, purchase order, goods receipt, invoice matching, posting request, dan reporting menjadi issue terpisah. Coder tidak membuat ledger baru bila accounting extension sudah tersedia. Reviewer menguji segregation of duties, period lock, immutability, dan reconciliation.

Contoh 2 - AWCMS-Mini: modul surat tugas

Derived application menambah duty-order, approval, numbering, attachment, dan report module pada application registry. Identity, workflow engine, audit, RLS, form draft, dan PDF infrastructure digunakan ulang. Security test membuktikan pengguna unit lain tidak dapat membaca surat tugas tenant berbeda.

Contoh 3 - AWCMS-Micro: website berita full-online

Agent menggunakan modul content/news yang tersedia, R2 untuk media, SEO/Open Graph, social share, Turnstile untuk form, dan Cloudflare profile. Fokus E2E mencakup publikasi, preview, responsive rendering, accessibility, dan provider outage tanpa menggagalkan penyimpanan draft.

Contoh 4 - Perbaikan vulnerability idempotency

Issue hanya memperbaiki request hash yang belum terikat resource ID. Coder menambahkan adversarial integration test: key sama pada resource berbeda harus menghasilkan conflict dan resource kedua tidak berubah. Reviewer melakukan repo-wide grep untuk menemukan kelas bug serupa; temuan unrelated dibuat issue baru.

Contoh 5 - Integrasi WhatsApp/email/AI

Transaksi utama hanya menulis data dan outbox dalam satu transaction. Worker memanggil provider setelah commit, menggunakan retry bounded, idempotency, circuit breaker, redacted logging, dan provider health check. Kegagalan provider tidak membatalkan transaksi operasional.

16. Pelajaran dari Pengembangan AWCMS-Mini

- 1. Base harus dinyatakan selesai secara eksplisit.** Tanpa batas ini, agent cenderung terus membangun ulang tenant, auth, access, logging, dan deployment.
- 2. Skill repository mengurangi improvisasi agent.** Pola migration, endpoint, event, idempotency, ABAC, audit, testing, dan deployment menjadi konsisten.
- 3. Reviewer independen menemukan kelas bug yang tidak terlihat coder.** Terutama idempotency, module boundary, provider transaction, RLS, dan generated artifact drift.
- 4. Security finding harus menjadi regression test.** Review naratif saja tidak cukup untuk mencegah bug berulang.
- 5. Generated inventory dan contract checks mencegah documentation drift.** CI harus menjalankan check yang sama dengan local quality gate.
- 6. Derived application membutuhkan compatibility manifest.** Valid hari ini tidak berarti kompatibel setelah base naik versi.
- 7. Review dan testing memang lebih lama daripada coding awal.** Optimasi dilakukan melalui issue lebih kecil, targeted test, parallel read-only review, dan automation - bukan dengan mengurangi quality gate.
- 8. Agent harus jujur terhadap kegagalan.** Command yang gagal, test flaky, environment limitation, dan residual risk harus dilaporkan.

17. Standar dan Kepatuhan

Pemetaan berikut digunakan sebagai panduan implementasi praktis, bukan pernyataan sertifikasi. Kepatuhan formal memerlukan penetapan scope, kebijakan organisasi, bukti operasional, audit, dan penilaian hukum yang sesuai.

Acuan	Penerapan dalam pedoman
ISO/IEC 27001 dan 27002	Secure development lifecycle, access control, logging, change control, separation of duties, incident readiness.
ISO/IEC 27005	Risk assessment per issue, severity, treatment, accepted residual risk.
ISO/IEC 27034	Application security, threat modeling, secure coding, verification, security audit.
ISO/IEC 20000	Change, release, deployment, configuration, incident, dan service operation.
ISO 22301	Business continuity, backup, restore, DR drill, rollback, continuity saat provider gagal.

Acuan	Penerapan dalam pedoman
ISO/IEC 27701	Privacy governance, minimization, access evidence, retention, deletion, processor/provider control.
ISO/IEC 27017 dan 27018	Cloud security, tenant isolation, cloud provider, secret management, perlindungan PII di cloud.
OWASP Top 10, ASVS, API Security Top 10	Auth, access, input validation, IDOR/BOLA, SSRF, injection, logging, rate limiting.
NIST CSF dan CIS Controls	Identify, Protect, Detect, Respond, Recover; hardening dan operational evidence.
UU No. 27 Tahun 2022 tentang PDP	Data minimization, tujuan pemrosesan, kontrol akses, perlindungan data pribadi, incident handling.
UU ITE sebagaimana terakhir diubah dengan UU No. 1 Tahun 2024	Keandalan sistem elektronik, keamanan transaksi/informasi, bukti elektronik, penggunaan sistem yang bertanggung jawab.
PP No. 71 Tahun 2019 tentang PSTE	Keandalan, keamanan, tata kelola PSE, audit trail, continuity, dan perlindungan sistem elektronik.

17.1 Minimum privacy review

- ☐ Tujuan dan dasar pemrosesan data jelas.
- ☐ Data yang dikumpulkan minimum dan proporsional.
- ☐ Peran yang dapat mengakses data terdokumentasi dan diuji.
- ☐ Data sensitif dimasking pada response, log, audit, dan event.
- ☐ Retensi, archive, purge, legal hold, dan backup deletion dipahami.
- ☐ Provider/cloud processor dan lokasi pemrosesan dievaluasi.
- ☐ Incident response dan notifikasi internal tersedia.

18. Penutup

Keberhasilan penggunaan agent tidak diukur dari banyaknya kode yang dihasilkan, tetapi dari perubahan yang benar, kecil, dapat ditelusuri, aman, teruji, terdokumentasi, dan dapat dioperasikan. Keluarga AWCMS menggunakan agent sebagai bagian dari sistem engineering yang memiliki kontrak, bukti, dan human approval.

Pedoman singkat

Klasifikasikan dengan benar. Gunakan base yang sudah ada. Kerjakan issue secara atomic. Satu writer per branch. Review dan audit secara independen. Buktikan dengan test. Merge dan deploy hanya melalui gate.

Lampiran A. Format Laporan Agent

Summary:
 - Tujuan issue dan hasil implementasi.

Files changed:
 - path/file: alasan perubahan.

Commands run:
 - command: PASS/FAIL dan ringkasan.

Test results:
 - unit / integration / contract / security / E2E / performance.

Security notes:
 - auth, tenant, ABAC, RLS, idempotency, audit, masking, provider boundary.

Documentation updates:
 - docs, ADR, OpenAPI, AsyncAPI, inventory, i18n, config, changeset.

Deployment/rollback notes:
 - migration, environment, backup, rollback, monitoring.

Remaining limitations:
 - residual risk, untested path, environment limitation, follow-up issue.

Next recommended step:
 - review, security audit, merge, deployment, atau issue berikutnya.

Lampiran B. Matriks Pemilihan Skill

Kebutuhan	Skill/pola
Issue end-to-end	awcms-*implement-issue / coder
Modul baru	awcms-*new-module
Migration/RLS	awcms-*new-migration
REST/OpenAPI	awcms-*new-endpoint
Event/AsyncAPI	awcms-*new-event
ABAC/RLS	awcms-*abac-guard
Idempotency	awcms-*idempotency
Audit/redaction	awcms-*audit-log / sensitive-data
UI/i18n/wizard	awcms-*ui-screen / i18n / wizard-form
Testing/E2E	awcms-*testing / browser-test
PR review	awcms-*pr-review / reviewer
Security review	awcms-*security-review / security-auditor
Performance/integration hardening	awcms-*performance / integration
Preflight/deploy/release	awcms-*production-preflight / deploy / release
Inventory/snapshot maintenance	awcms-*repo-inventory / github-snapshot

Referensi

- [1] AWCMS-Mini README - arsitektur, status base, stack, prinsip, dan paket dokumen. - <https://github.com/ahliweb/awcms-mini/blob/main/README.md>

- [2] AWCMS-Mini AGENTS.md - kontrak kerja agent, guardrail, skill, command, dan struktur repository. - <https://github.com/ahliweb/awcms-mini/blob/main/AGENTS.md>
- [3] AWCMS-Mini CONTRIBUTING.md - alur kontribusi, branch, test, commit, Definition of Done. - <https://github.com/ahliweb/awcms-mini/blob/main/CONTRIBUTING.md>
- [4] Panduan Implementasi Aplikasi Turunan - extension layer, application registry, compatibility manifest, dan alur sembilan langkah. - <https://github.com/ahliweb/awcms-mini/blob/main/docs/awcms-mini/derived-application-guide.md>
- [5] AWCMS-Mini Project Skills - katalog skill dan subagent. - <https://github.com/ahliweb/awcms-mini/blob/main/.claude/skills/README.md>
- [6] Skill awcms-mini-implement-issue. - <https://github.com/ahliweb/awcms-mini/blob/main/.claude/skills/awcms-mini-implement-issue/SKILL.md>
- [7] Skill awcms-mini-pr-review. - <https://github.com/ahliweb/awcms-mini/blob/main/.claude/skills/awcms-mini-pr-review/SKILL.md>
- [8] Skill awcms-mini-security-review. - <https://github.com/ahliweb/awcms-mini/blob/main/.claude/skills/awcms-mini-security-review/SKILL.md>
- [9] Undang-Undang Republik Indonesia Nomor 27 Tahun 2022 tentang Perlindungan Data Pribadi.
- [10] Undang-Undang Republik Indonesia Nomor 1 Tahun 2024 tentang Perubahan Kedua atas Undang-Undang Informasi dan Transaksi Elektronik.
- [11] Peraturan Pemerintah Republik Indonesia Nomor 71 Tahun 2019 tentang Penyelenggaraan Sistem dan Transaksi Elektronik.
- [12] ISO/IEC 27001:2022; ISO/IEC 27002:2022; ISO/IEC 27005; ISO/IEC 27034; ISO/IEC 27701; ISO/IEC 27017; ISO/IEC 27018; ISO/IEC 20000; ISO 22301.
- [13] OWASP Top 10, OWASP ASVS, OWASP API Security Top 10, NIST Cybersecurity Framework, dan CIS Controls.